

# Directorio de Médicos Especialistas — Documentación Técnica

**Cliente:** Ministerio de Educación de Guatemala · **Curso:** CC3106 Responsable AI, UVG **Equipo:** 4 personas · **Alcance:** Ciudad de Guatemala, zonas 1–18 **Repositorio:** Functions + UI + pruebas + CI · **Fecha de esta versión:** <completar al entregar>

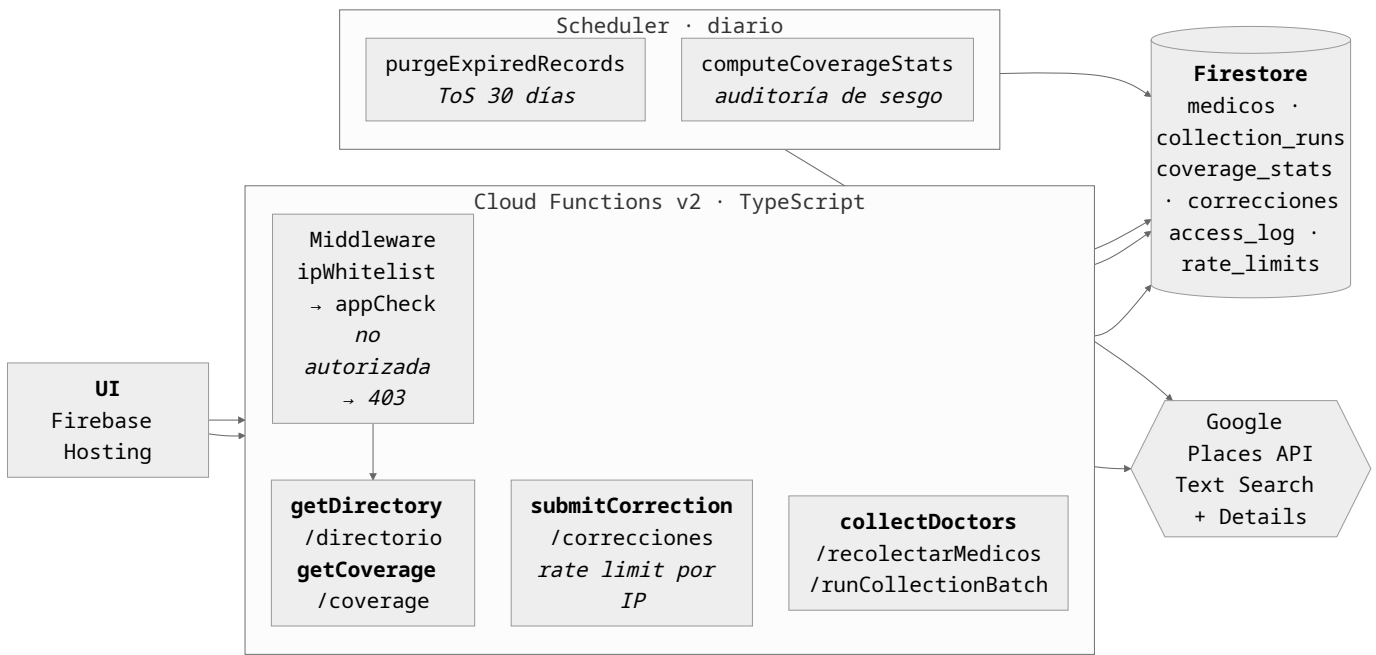
Los marcadores <pendiente> indican números que solo existen tras la recolección en producción. No se rellenan con estimaciones: inventar cifras en un documento sobre manejo responsable de datos contradiría el propio entregable.

## 1. Qué es este sistema

Un directorio de médicos especialistas construido sobre Google Places API, con API paginada protegida por IP whitelist y UI de consulta.

El sistema no se limita a recolectar: **mide y publica sus propios sesgos** (§8), implementa los límites de los Términos de Servicio de Google en código y no en prosa (§7), y expone un mecanismo de corrección y remoción para las personas cuyos datos aparecen sin haberlo consentido (§11). La IP whitelist se implementa como se pide, y se documenta por qué no basta (§6).

## 2. Arquitectura



**Stack:** TypeScript · Firebase Functions v2 · Firestore · Google Places API · Firebase Hosting. El 90 % del desarrollo corre contra el emulador de Firebase; producción se usa solo para las pruebas finales y la demo, porque cada invocación desplegada tiene costo real.

## Cloud Functions

| Función                       | Ruta / disparador       | Responsabilidad                                                                                                 |
|-------------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------|
| collectDoctors                | GET /recolectarMedicos  | Búsqueda puntual: keyword + zona + especialidad → Places → Firestore. Tope <b>20 resultados</b> por invocación. |
| collectDoctors                | GET /runCollectionBatch | Recorre la matriz de 576 keywords con cursor persistente, en lotes de 1–10.                                     |
| getDirectory                  | GET /directorio         | Paginado con filtros. Excluye suppressed y vencidos.                                                            |
| getCoverage                   | GET /coverage           | Sirve coverage_stats precalculado para el heatmap.                                                              |
| submitCorrection              | POST /correcciones      | Corrección o remoción. <b>Sin IP whitelist</b> (debe ser público); rate limit por IP.                           |
| purgeExpiredRecordsScheduled  | Diario                  | Refresca o purga documentos vencidos. Cumplimiento ToS.                                                         |
| computeCoverageStatsScheduled | Diario                  | Recalcula la matriz zona × especialidad.                                                                        |

## 3. Modelo de datos

place\_id es la clave del documento en medicos : es el identificador estable de Google y convierte la reinserción en un upsert idempotente, así que la deduplicación es una propiedad del esquema, no un paso de limpieza.

```
medicos/{place_id}
  nombre, direccion, telefono|null, sitio_web|null, zona, lat|null, lng|null
  especialidad          # efectiva – el campo por el que filtra /directorio
  especialidad_raw       # la especialidad con la que se buscó (trazabilidad)
  especialidad_normalizada # derivada del nombre por reglas | null
  confidence             # 0.9 si hubo match, 0 si no
  missing_fields: string[] # ausencias explícitas, nunca cadenas vacías
  fecha_recoleccion, expires_at, run_id, keyword_usado
  suppressed: boolean     # true tras una solicitud de remoción
```

Colecciones de apoyo: collection\_runs (telemetría por corrida), coverage\_stats (matriz precalculada), correcciones (cola de revisión), access\_log (200/403/429), rate\_limits (ventana fija de 60 s), collection\_progress/state (cursor compartido).

### Normalización de especialidades

Google Places **no tiene un campo de especialidad médica**. Lo único disponible es el nombre del negocio como texto libre. Ante eso se evaluaron dos opciones y se eligió la primera:

- Reglas determinísticas (implementada).** Un diccionario de raíces por especialidad escanea el nombre sin distinguir acentos ni género/número. Si hay coincidencia, especialidad\_normalizada se llena con confianza 0.9; si no, queda null con confianza 0. **Nunca se inventa una especialidad.**
- LLM (descartada).** Exigiría un set de evaluación etiquetado a mano (≥100 registros) y presupuesto de inferencia no contemplado. El criterio de descarte no fue solo costo: una especialidad alucinada puede derivar en daño al paciente, y ese error pesa más que la cobertura adicional que un modelo aportaría.

/directorio filtra por especialidad = especialidad\_normalizada ?? especialidad\_raw : lo que el lugar aparenta ser gana sobre lo que buscamos. Una "Clínica Pediátrica" encontrada con un keyword de cardiología aparece bajo pediatría, no bajo cardiología. El fallback a la especialidad buscada es deliberado — cuando el nombre no revela nada, esa es la única señal disponible, y preferimos una señal débil pero declarada a esconder el registro de todo filtro.

## 4. Recolección de datos

**Patrón de keyword:** "<sufijo> <especialidad> <zona> Guatemala", con 4 sufijos (médico, doctor, clínica, consultorio) porque Google Maps no tiene nomenclatura consistente para consultorios médicos.

**Matriz:** 8 especialidades × 4 sufijos × 18 zonas = **576 combinaciones**.

**Cobertura balanceada obligatoria.** Se ejecuta el mismo número de búsquedas en cada zona sin importar cuántos resultados devuelva. Concentrar el esfuerzo donde "sí hay resultados" contaminaría la auditoría de sesgo: mediríamos nuestro propio criterio de búsqueda en lugar de la realidad de la fuente. Esta es la restricción metodológica central del proyecto.

Cada invocación consume 1 llamada de Text Search + hasta 20 de Place Details (teléfono y sitio web no vienen en Text Search), y escribe un documento en `collection_runs` con keyword, zona, resultados nuevos vs. duplicados y costo estimado. `runCollectionBatch` avanza un cursor compartido en Firestore, de modo que cualquier integrante continúa donde otro se quedó sin repetir búsquedas ni saltarse zonas.

---

## 5. API

GET `/directorio` — paginado, con filtros opcionales.

| Parámetro                 | Tipo            | Default | Límite                           |
|---------------------------|-----------------|---------|----------------------------------|
| <code>page</code>         | entero $\geq 1$ | 1       | —                                |
| <code>pageSize</code>     | entero $\geq 1$ | 20      | 50 (se recorta, no se rechaza)   |
| <code>especialidad</code> | string          | —       | filtra por especialidad efectiva |
| <code>zona</code>         | string          | —       | <code>zona 1 ... zona 18</code>  |

Respuesta: `{ results, page, pageSize, hasMore }`. Entradas inválidas caen al default en vez de devolver 400 — un `pageSize=abc` no debe romper la UI.

Los filtros de `suppressed` y `expires_at` se aplican **en memoria**, no en la consulta de Firestore, para no exigir un índice compuesto por cada combinación de filtros. Como eso podría dejar una página a medio llenar, la consulta sobre-lee con duplicación progresiva del límite (hasta 5 rondas) antes de recurrir a leer todo.

---

## 6. Seguridad y modelo de amenazas

La IP whitelist se implementa como middleware, primer paso de la request: si la IP no está autorizada devuelve **403 sin ejecutar ninguna lógica adicional**. Toda decisión (200/403/429) queda en `access_log`.

La IP se deriva del proxy de confianza de Cloud Functions. **No se confía en X-Forwarded-For crudo**, que el cliente puede falsificar; hay un test automatizado que envía el header falso y exige 403.

**Por qué la whitelist no basta:**

| Debilidad                                               | Impacto                          | Mitigación aplicada                          |
|---------------------------------------------------------|----------------------------------|----------------------------------------------|
| <code>X-Forwarded-For</code> falsificable si se lee mal | Bypass total                     | IP del proxy de confianza + test de spoofing |
| Es ubicación de red, no autenticación                   | Cualquiera dentro de la IP entra | Rate limit por IP + auditoría                |
| IPs dinámicas / red móvil                               | Falsos negativos, acceso roto    | Limitación operativa documentada             |
| No frena abuso desde IP autorizada                      | Extracción masiva del dataset    | Rate limit + tope de 50 por página           |

**Defensa en profundidad:** Firebase App Check en la UI, rate limit por IP en `/correcciones`, `access_log` auditable, y API key restringida por IP en la consola de GCP. La key vive en variables de entorno y nunca en el código fuente.

**Cierre del hueco de configuración.** Que `APP_CHECK_ENFORCE` existiera en el código no garantizaba nada: si nadie la ponía en `true` al desplegar, quedaba inerte sin más señal que un log que nadie revisa. `firebase.json` corre ahora un script de predeploy que **aborta el deploy** si el proyecto destino no es de emulador y falta `APP_CHECK_ENFORCE=true`. Declarar la variable dejó de ser suficiente; el pipeline lo hace cumplir.

---

## 7. Cumplimiento de los Términos de Servicio

Los ToS de Google Maps Platform permiten almacenar `place_id` indefinidamente, pero el resto del contenido de Places solo puede cachearse de forma temporal, con un máximo de **30 días calendario consecutivos**. Un proyecto que guarda nombres y teléfonos "para siempre porque es académico" está en incumplimiento.

Cada documento lleva `expires_at = fecha_recoleccion + 30 días`, y la purga diaria hace:

- **Refresco:** re-consulta por `place_id`, reemplaza el contenido y renueva la ventana.
- **Purga:** elimina el contenido y conserva `place_id` más un `purge_reason` — `suppressed`, `no_api_key`, `not_found_in_places` o `refresh_error`. La distinción importa: "Google confirmó que cerró" y "no teníamos la key configurada" producirían documentos idénticos si no se registrara el motivo, y el segundo caso es un problema de configuración disfrazado de dato.

Un registro `suppressed` **nunca se refresca**, se purga directo: refrescar a alguien que pidió su remoción desharía la remoción. La purga borra también `especialidad`, `especialidad_normalizada` y `confidence`, porque los tres derivan del nombre del negocio y por lo tanto son contenido de Places sujeto a la misma ventana.

---

## 8. Auditoría de cobertura y sesgo

Con los mismos datos ya recolectados (costo de API adicional  $\approx 0$ ) se calcula por celda zona  $\times$  especialidad: búsquedas ejecutadas, resultados únicos, % con teléfono y % con sitio web.

El detalle metodológico, los datos y el hallazgo están en `docs/auditoria-cobertura.md`. Una decisión de método merece mención aquí: `searches_run` se cuenta desde `collection_runs`, no desde `medicos`. Si se contara desde los resultados, una zona buscada diez veces sin encontrar nada y una zona nunca buscada se verían idénticas —cero registros— y esa distinción es justamente lo que la auditoría existe para capturar.

---

## 9. Costos

Text Search (~\$0.032) y Place Details (~\$0.017) tienen tarifas distintas; sumarlas a una sola subestimaba el gasto, así que `collection_runs` las contabiliza por separado.

| Métrica                       | Valor       |
|-------------------------------|-------------|
| Gasto total                   | <pendiente> |
| Registros únicos              | <pendiente> |
| <b>Costo por médico único</b> | <pendiente> |
| Tasa de duplicados            | <pendiente> |

Controles preventivos: alertas de billing al 50 % y 90 % del presupuesto, cuota máxima diaria de llamadas en la consola de APIs, y desarrollo en emulador. Cada integrante usa su propio proyecto GCP y responde por su propio gasto.

---

## 10. Pruebas y CI

34 pruebas de integración corren contra el emulador de Firebase en GitHub Actions, con costo de API cero. Cubren: IP no autorizada  $\rightarrow$  403; `X-Forwarded-For` falsificado  $\rightarrow$  403; tope de `pageSize`; documento vencido ausente de `/directorio`; la purga conserva `place_id` y elimina contenido; un registro `suppressed` no reaparece tras re-recolección; reinserción del mismo `place_id` no duplica; y un recorrido end-to-end que va de la recolección a la remoción.

Correr la suite completa contra producción consumiría cuota real, así que ese paso es un checklist manual previo a la demo, documentado en el README.

---

## 11. Postura ética

- **ToS:** la ventana de 30 días está implementada en código, no declarada en prosa.
- **No fabricación:** no se agregan ni infieren datos ausentes. Los campos faltantes se guardan como `null` con `missing_fields`, y la UI muestra "No reportado en la fuente" — nunca un espacio en blanco que parezca dato. El campo `sitio_web` puede estar vacío o apuntar a una clínica en lugar de a un sitio propio; se documenta como viene.
- **Sesgo de fuente:** publicamos la auditoría de cobertura y su hallazgo, incluso cuando el hallazgo debilita la utilidad aparente del producto.
- **Uso previsto y prohibido:** el directorio es referencia informativa, **no validación de credenciales médicas**. Se declara en la UI y en la Data Card.

- **Frescura:** `fecha_recoleccion` acompaña todo dato mostrado o exportado, con su antigüedad en días.
  - **Derechos de terceros:** los médicos listados no consintieron aparecer aquí; que su información sea pública en Google Maps no equivale a consentimiento para redistribuirla. El mecanismo de corrección y remoción es operativo y visible en la UI, no escondido en un pie de página. Una remoción sobrevive a recolecciones futuras.
  - **Alcance:** los datos no se redistribuyen como producto independiente. El alcance es estrictamente académico; en producción real haría falta un acuerdo comercial con Google.
- 

## 12. Limitaciones conocidas

---

La limitación central es que **la cobertura refleja presencia digital en Google Maps, no oferta médica**: una celda vacía significa "Google no lo tiene", nunca "ahí no hay médicos". Le siguen que la especialidad es inferida o asumida pero nunca verificada, que la whitelist protege ubicación de red y no identidad, y que el dato puede tener hasta 30 días de antigüedad. El inventario completo está en `docs/data-card.md` §5 y el detalle del sesgo en `docs/auditoria-cobertura.md`.